

Class 13: RNAseq analysis with DESeq

David Tin (PID:A17785446)

Table of contents

Background	1
Data Import	1
DESeq analysis	7
Volcano plot	8
Save our results to date	12
Pathway analysis with R and BioConductor	12

Background

Today we're going to do an RNA-seq analysis of a data set on the common glucocorticoid steroid dexamethasone (Dex), and we'll use DESeq for this analysis.

Data Import

Let's read the `count` data and `metadata` about this experiment setup from the supplied CSV files:

```
# Complete the missing code
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <- read.csv("airway_metadata.csv")
```

Have a peek at the data:

```
head(counts)
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG000000000003	723	486	904	445	1170
ENSG000000000005	0	0	0	0	0
ENSG000000000419	467	523	616	371	582
ENSG000000000457	347	258	364	237	318
ENSG000000000460	96	81	73	66	118
ENSG000000000938	0	0	1	0	2

	SRR1039517	SRR1039520	SRR1039521
ENSG000000000003	1097	806	604
ENSG000000000005	0	0	0
ENSG000000000419	781	417	509
ENSG000000000457	447	330	324
ENSG000000000460	94	102	74
ENSG000000000938	0	0	0

And the metadata that tells us what is actually in the columns of our `counts` object:

```
metadata
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871
7	SRR1039520	control	N061011	GSM1275874
8	SRR1039521	treated	N061011	GSM1275875

Q1. How many genes are in this dataset?

There are 38694 genes in this dataset

Q2. How many 'control' cell lines do we have?

```
sum(metadata$dex=="control")
```

```
[1] 4
```

```
ncol(counts)
```

```
[1] 8
```

```
colnames(counts)
```

```
[1] "SRR1039508" "SRR1039509" "SRR1039512" "SRR1039513" "SRR1039516"  
[6] "SRR1039517" "SRR1039520" "SRR1039521"
```

- Find the “control” columns in our `counts` object
- Extract just the “control” column values for all genes
- Calculate the average value per gene in these “control” columns

```
control.inds <- metadata$dex == "control"  
control.counts <- counts[,control.inds]  
control.mean <- rowMeans(control.counts)
```

The same but for treated columns

```
treated.mean <- rowMeans( counts[, metadata$dex == "treated"])
```

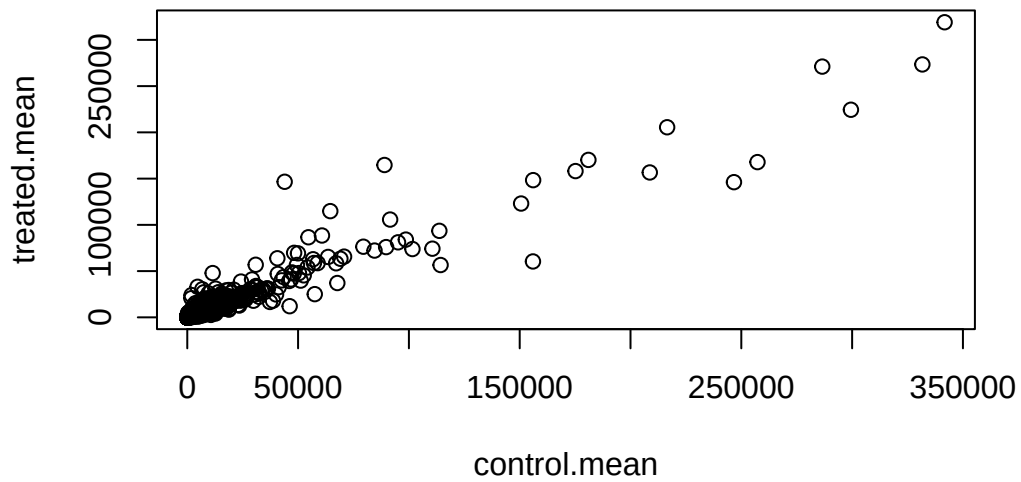
For book-keeping let’s store these together as a new object called `meancounts`

```
meancounts <- data.frame(control.mean, treated.mean)  
head(meancounts)
```

	control.mean	treated.mean
ENSG000000000003	900.75	658.00
ENSG000000000005	0.00	0.00
ENSG000000000419	520.50	546.00
ENSG000000000457	339.75	316.50
ENSG000000000460	97.25	78.75
ENSG000000000938	0.75	0.00

Make a plot of `control.mean` vs `treated.mean`

```
plot(control.mean, treated.mean)
```

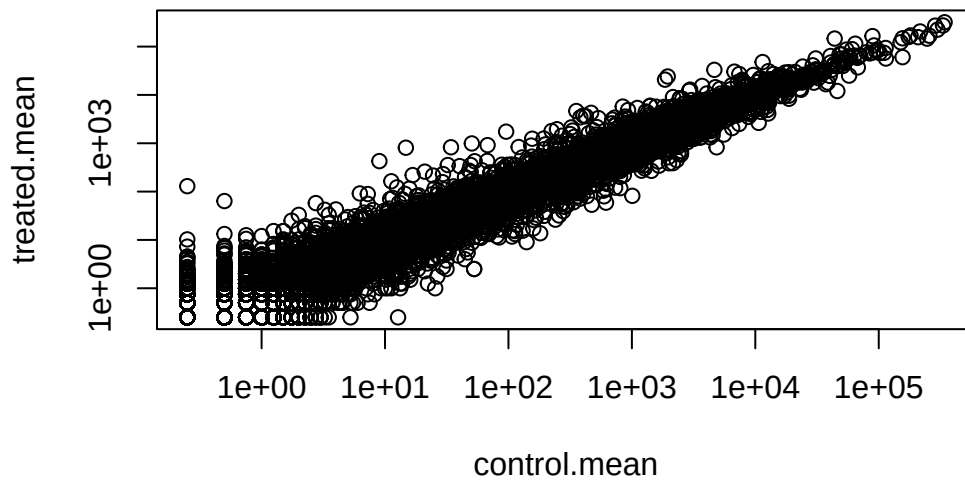


Our count data is highly skewed and when we see a pattern like this plot it screams log transform

```
plot(meancounts, log="xy")
```

```
Warning in xy.coords(x, y, xlabel, ylabel, log): 15032 x values <= 0 omitted  
from logarithmic plot
```

```
Warning in xy.coords(x, y, xlabel, ylabel, log): 15281 y values <= 0 omitted  
from logarithmic plot
```



We most often use log2 transform for this kind of data in bioinformatics because it makes the interpretation easier

```
#Treated / Control
```

```
log2( 20 / 20 )
```

```
[1] 0
```

```
log2( 40 / 20 )
```

```
[1] 1
```

```
log2( 10 / 20 )
```

```
[1] -1
```

```
log2( 80 / 20 )
```

```
[1] 2
```

We call this fraction the “**log2 fold change**” as it tells us how much more or less gene expression we have in units of doubling, etc.

Let’s calculate the log2 fold change for our `treated.mean` and `control.mean` counts and call this `log2fc`.

```
meancounts$log2fc <- log2(meancounts$treated.mean /
                          meancounts$control.mean)
head(meancounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000005	0.00	0.00	NaN
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000938	0.75	0.00	-Inf

A common “rule of thumb” threshold for calling a gene “up regulated” or “down regulated” is a log2 fold-change value of +2 or -2 (or greater)

```
zero.vals <- which(meancounts[,1:2]==0, arr.ind=TRUE)

to.rm <- unique(zero.vals[,1])
mycounts <- meancounts[-to.rm,]
head(mycounts)
```

	control.mean	treated.mean	log2fc
ENSG000000000003	900.75	658.00	-0.45303916
ENSG000000000419	520.50	546.00	0.06900279
ENSG000000000457	339.75	316.50	-0.10226805
ENSG000000000460	97.25	78.75	-0.30441833
ENSG000000000971	5219.00	6687.50	0.35769358
ENSG000000001036	2327.00	1785.75	-0.38194109

Q7. What is the purpose of the `arr.ind` argument in the `which()` function call above? Why would we then take the first column of the output and need to call the `unique()` function?

`arr.ind` tells `which()` to return the positions of matching values as row and column indices. `unique()` is used since rows may contain multiple zeros, so it removes duplicate row indices.

```
up.ind <- mycounts$log2fc > 2  
down.ind <- mycounts$log2fc < (-2)
```

Q8. Using the up.ind vector above can you determine how many up regulated genes we have at the greater than 2 fc level?

```
sum(up.ind)
```

```
[1] 250
```

DESeq analysis

Let's do this analysis properly and not forget about the significance of the differences

For this we will use the **DESeq2** package

```
library(DESeq2)
```

To run a DESeq analysis we need at least two inputs:

- countData i.e. are gene counts across different experiments
- colData i.e. our metadata about those count columns

```
dds <- DESeqDataSetFromMatrix(countData = counts,  
                              colData = metadata,  
                              design = ~dex)
```

converting counts to integer mode

Now we can run the DEseq analysis pipeline using this dds object that has all the inputs we need.

```
dds <- DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

```
res <- results(dds)
head(res)
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 6 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.194195	-0.3507030	0.168246	-2.084470	0.0371175
ENSG000000000005	0.000000	NA	NA	NA	NA
ENSG000000000419	520.134160	0.2061078	0.101059	2.039475	0.0414026
ENSG000000000457	322.664844	0.0245269	0.145145	0.168982	0.8658106
ENSG000000000460	87.682625	-0.1471420	0.257007	-0.572521	0.5669691
ENSG000000000938	0.319167	-1.7322890	3.493601	-0.495846	0.6200029
	padj				
	<numeric>				
ENSG000000000003	0.163035				
ENSG000000000005	NA				
ENSG000000000419	0.176032				
ENSG000000000457	0.961694				
ENSG000000000460	0.815849				
ENSG000000000938	NA				

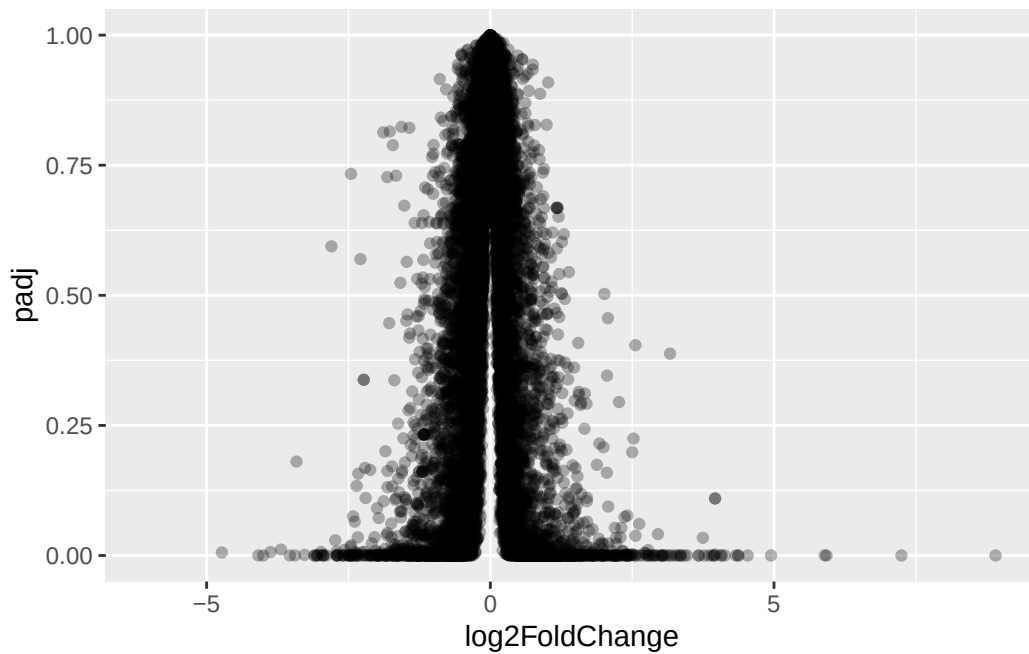
Volcano plot

This is a ubiquitous and common visualization for this type of data that puts the log fold change and the adjusted p-value together in one plot that people can get insight for what is going on in the whole dataset results.

```
library(ggplot2)
```

```
ggplot(res) +
  aes(log2FoldChange, padj) +
  geom_point(alpha=0.3)
```


Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).

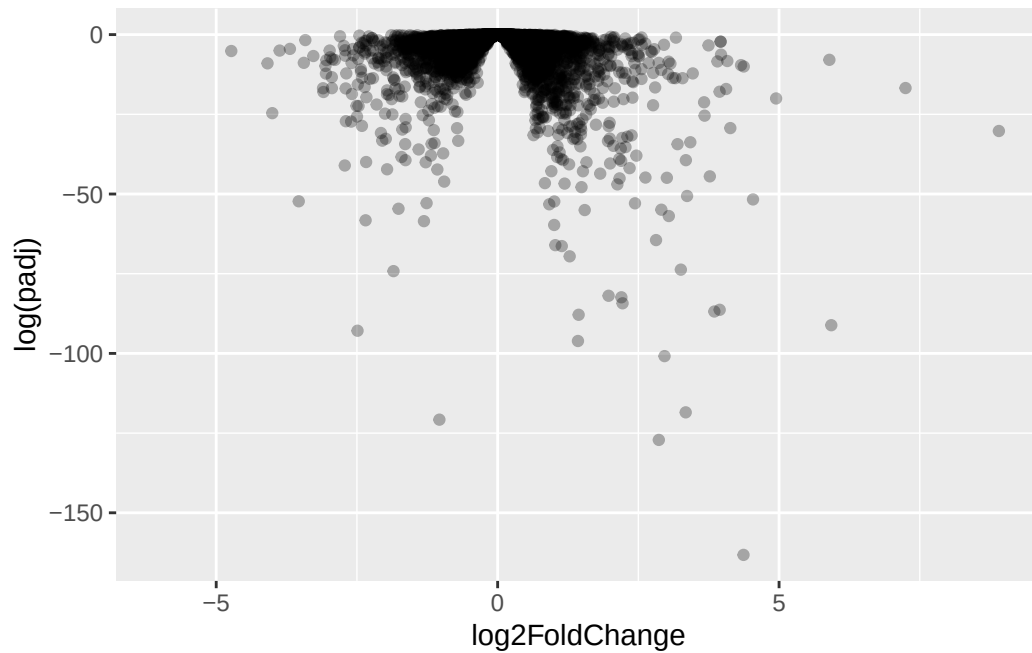


That plot is not very useful because we don't care about genes with high p-values, we want the very low values below our alpha threshold (e.g. 0.01)

Let's log the y-axis so we can see these genes/points more clearly

```
ggplot(res) +  
  aes(log2FoldChange, log(padj)) +  
  geom_point(alpha=0.3)
```

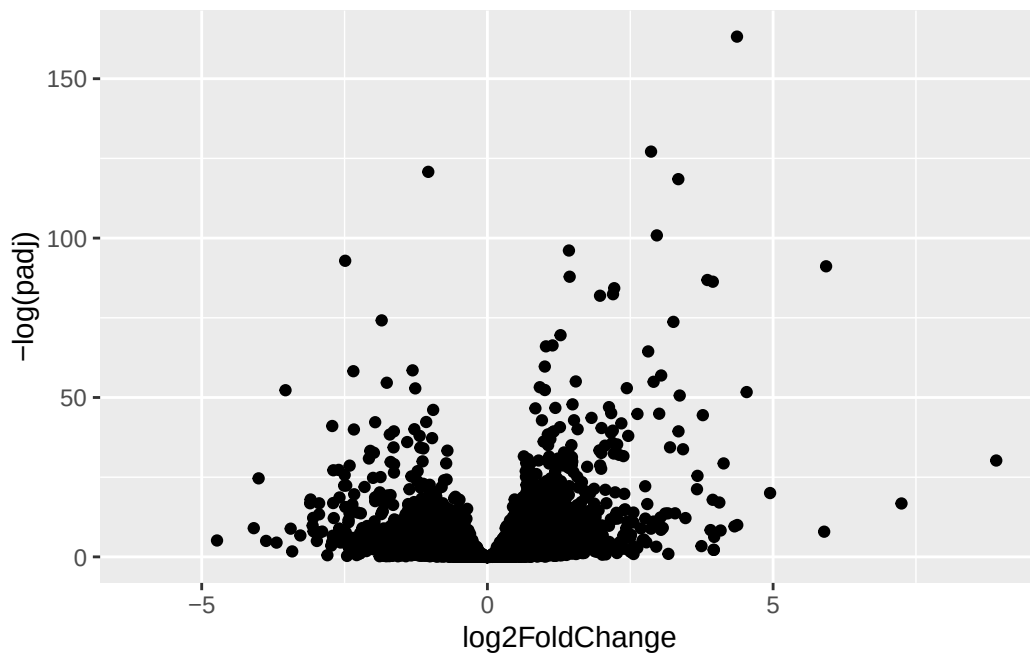
Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).



We need to flip the y-axis so our “volcano” is not upside down

```
ggplot(res) +  
  aes(log2FoldChange, -log(padj)) +  
  geom_point()
```

Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).



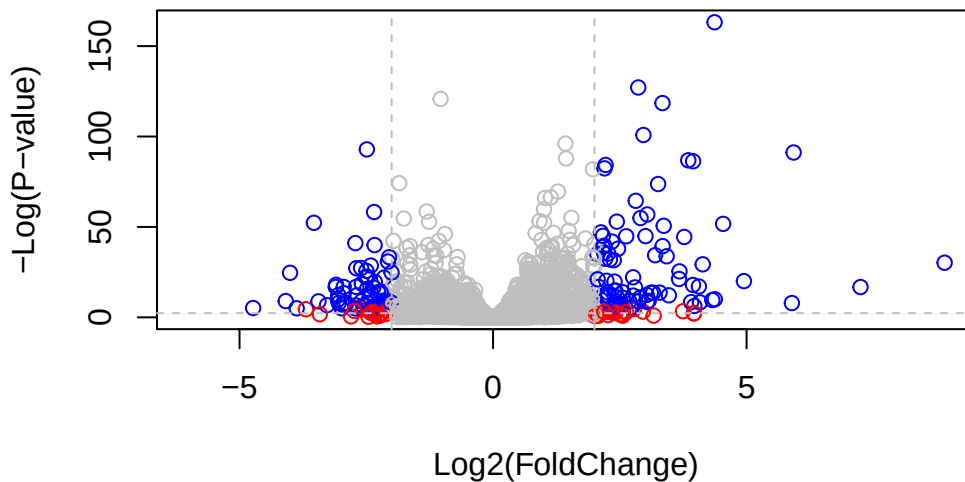
Q. Add annotation to this volcano plot including the log2 fold-change thresholds of +2 and -2 and the p-value threshold of 0.05. Also color up just the genes that meet both these thresholds.

```
# Setup our custom point color vector
mycols <- rep("gray", nrow(res))
mycols[ abs(res$log2FoldChange) > 2 ] <- "red"

inds <- (res$padj < 0.01) & (abs(res$log2FoldChange) > 2 )
mycols[ inds ] <- "blue"

# Volcano plot with custom colors
plot( res$log2FoldChange, -log(res$padj),
      col=mycols, ylab="-Log(P-value)", xlab="Log2(FoldChange)" )

# Cut-off lines
abline(v=c(-2,2), col="gray", lty=2)
abline(h=-log(0.1), col="gray", lty=2)
```



Save our results to date

```
write.csv(res, file="myresults.csv")
```

Pathway analysis with R and BioConductor

```
library(pathview)
```

```
#####
Pathview is an open source software package distributed under GNU General
Public License version 3 (GPLv3). Details of GPLv3 is available at
http://www.gnu.org/licenses/gpl-3.0.html. Particullary, users are required to
formally cite the original Pathview paper (not just mention it) in publications
or products. For details, do citation("pathview") within R.
```

The pathview downloads and uses KEGG data. Non-academic uses may require a KEGG

license agreement (details at <http://www.kegg.jp/kegg/legal.html>).
#####

```
library(gage)
```

```
library(gageData)
```

```
data(kegg.sets.hs)
```

```
# Examine the first 2 pathways in this kegg set for humans  
head(kegg.sets.hs, 2)
```

```
$`hsa00232 Caffeine metabolism`
```

```
[1] "10" "1544" "1548" "1549" "1553" "7498" "9"
```

```
$`hsa00983 Drug metabolism - other enzymes`
```

```
[1] "10" "1066" "10720" "10941" "151531" "1548" "1549" "1551"  
[9] "1553" "1576" "1577" "1806" "1807" "1890" "221223" "2990"  
[17] "3251" "3614" "3615" "3704" "51733" "54490" "54575" "54576"  
[25] "54577" "54578" "54579" "54600" "54657" "54658" "54659" "54963"  
[33] "574537" "64816" "7083" "7084" "7172" "7363" "7364" "7365"  
[41] "7366" "7367" "7371" "7372" "7378" "7498" "79799" "83549"  
[49] "8824" "8833" "9" "978"
```

```
foldchanges = res$log2FoldChange  
names(foldchanges) = res$entrez  
head(foldchanges)
```

```
[1] -0.35070302 NA 0.20610777 0.02452695 -0.14714205 -1.73228897
```

```
# Get the results  
keggres = gage(foldchanges, gsets=kegg.sets.hs)  
attributes(keggres)
```

```
$names
```

```
[1] "greater" "less" "stats"
```

```
head(keggres$less)
```

	p.geomean	stat.mean	p.val	q.val
hsa00232 Caffeine metabolism	NA	NaN	NA	NA
hsa00983 Drug metabolism - other enzymes	NA	NaN	NA	NA
hsa01100 Metabolic pathways	NA	NaN	NA	NA
hsa00230 Purine metabolism	NA	NaN	NA	NA
hsa05340 Primary immunodeficiency	NA	NaN	NA	NA
hsa04514 Cell adhesion molecules (CAMs)	NA	NaN	NA	NA

	set.size	expl
hsa00232 Caffeine metabolism	0	NA
hsa00983 Drug metabolism - other enzymes	0	NA
hsa01100 Metabolic pathways	0	NA
hsa00230 Purine metabolism	0	NA
hsa05340 Primary immunodeficiency	0	NA
hsa04514 Cell adhesion molecules (CAMs)	0	NA

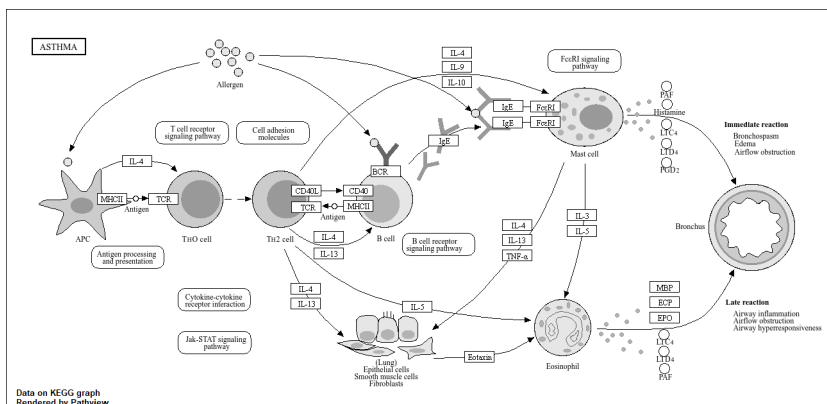
```
pathview(gene.data=foldchanges, pathway.id="hsa05310")
```

Warning: None of the genes or compounds mapped to the pathway!
 Argument gene.idtype or cpd.idtype may be wrong.

'select()' returned 1:1 mapping between keys and columns

Info: Working in directory C:/Users/david/Desktop/bimm143/class13

Info: Writing image file hsa05310.pathview.png



```
# Look at the first three down (less) pathways
head(keggres$less, 3)
```

	p.geomean	stat.mean	p.val	q.val
hsa00232 Caffeine metabolism	NA	NaN	NA	NA
hsa00983 Drug metabolism - other enzymes	NA	NaN	NA	NA
hsa01100 Metabolic pathways	NA	NaN	NA	NA

	set.size	expl
hsa00232 Caffeine metabolism	0	NA
hsa00983 Drug metabolism - other enzymes	0	NA
hsa01100 Metabolic pathways	0	NA

```
pathview(gene.data=foldchanges, pathway.id="hsa05310")
```

Warning: None of the genes or compounds mapped to the pathway!
Argument gene.idtype or cpd.idtype may be wrong.

'select()' returned 1:1 mapping between keys and columns

Info: Working in directory C:/Users/david/Desktop/bimm143/class13

Info: Writing image file hsa05310.pathview.png

Q12. Can you do the same procedure as above to plot the pathview figures for the top 2 down-regulated pathways?

```
pathview(gene.data=foldchanges, pathway.id="hsa00232")
```

Warning: None of the genes or compounds mapped to the pathway!
Argument gene.idtype or cpd.idtype may be wrong.

'select()' returned 1:1 mapping between keys and columns

Info: Working in directory C:/Users/david/Desktop/bimm143/class13

Info: Writing image file hsa00232.pathview.png

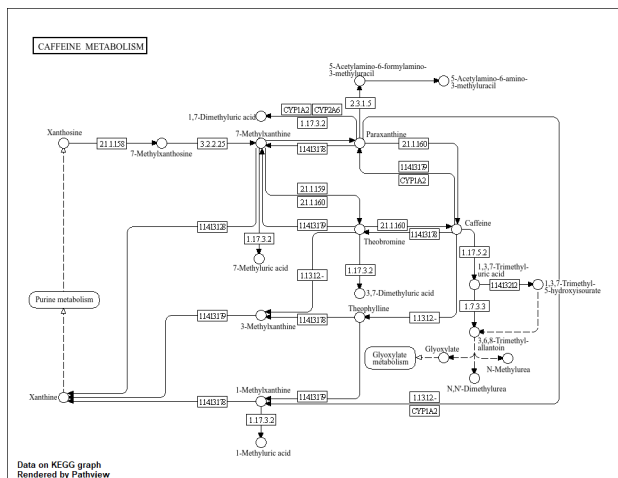
```
pathview(gene.data=foldchanges, pathway.id="hsa00983")
```

Warning: None of the genes or compounds mapped to the pathway!
Argument gene.idtype or cpd.idtype may be wrong.

'select()' returned 1:1 mapping between keys and columns

Info: Working in directory C:/Users/david/Desktop/bimm143/class13

Info: Writing image file hsa00983.pathview.png



Azathioprine & 6-Mercaptopurine



```

graph TD
    Irinotecan[Irinotecan (prodrug)] --> C77364[C77364]
    C77364 --> APC[APC (inactive)]
    APC --> NFC[NFC (inactive)]
    NFC --> SN38[SN-38 (active)]
    SN38 --> SN38G[SN-38G (inactive)]
    SN38G --> Excretion[excretion]
    Irinotecan --> SN38
    C77364 --> SN38
    C77364 --> SN38G
  
```

[illegible]